The Journal of Systems and Software 224 (2025) 112381

Contents lists available at ScienceDirect

The Journal of Systems & Software

journal homepage: www.elsevier.com/locate/jss

Pytester: Deep reinforcement learning for text-to-testcase generation ☆

Wannita Takerngsaksiri [a], Rujikorn Charakorn [b], Chakkrit Tantithamthavorn [a,*], Yuan-Fang Li [a]

[a] Monash University, Australia
[b] Vidyasirimedhi Institute of Science and Technology (VISTEC), Thailand

ARTICLE INFO

Keywords:
Text-to-testcase generation
Deep reinforcement learning

ABSTRACT

Test-driven development (TDD) is a widely-employed software development practice that mandates writing test cases based on a textual description *before* writing the actual code. While writing test cases is the centerpiece of TDD, it is time-consuming, expensive, and often shunned by developers. To address these issues associated with TDD, automated test case generation approaches have recently been investigated. Such approaches take source code as input, but not the textual description. Therefore, existing work does not fully support true TDD, as actual code is required to generate test cases. In addition, current deep learning-based test case generation approaches are trained with one learning objective, i.e., to generate test cases that are exactly matched with the ground-truth test cases. However, such approaches may limit the model's ability to generate different yet correct test cases. In this paper, we introduce PyTester, a Text-to-Testcase generation approach that can automatically generate syntactically correct, executable, complete, and effective test cases while being aligned with a given textual description. We evaluate PyTester on the public APPS benchmark dataset, and the results show that our Deep RL approach enables PyTester, a small language model, to outperform much larger language models like GPT3.5, StarCoder, and InCoder. Our findings suggest that future research could consider improving small over large LMs for better resource efficiency by integrating the SE domain knowledge into the design of reinforcement learning architecture.

1. Introduction

TDD is an agile software development practice (Beck et al., 2001) that advocates the creation of test cases before code development (Beck, 2022; Astels, 2003). The methodology comprises iterative steps of creating test cases, writing code to pass the test cases, and refactoring. Thus, developers can conceptualize the specification before coding, which increases developers' productivity (Erdogmus et al., 2005). Therefore, writing high-quality test cases is crucial for TDD, yet still time-consuming and complex.

Following TDD practices, developers may focus on writing test cases at different granularity levels (e.g., acceptance, integration, system, and unit test cases) depending on the details of the requirements (e.g., user stories, system description, class description, function description). However, current research only supports a few scenarios of TDD (e.g., generating acceptance test cases from natural language specifications). For example, Fischbach et al. (2023) and Wang et al. (2020) proposed an approach to generate acceptance test cases based on a use case specification. However, these approaches only generate test cases at the acceptance level for a given use case. While generating acceptance test cases will greatly help teams to check if all of the use cases are met, it does not guarantee functionality correctness for every method in the software.

Therefore, researchers proposed various automated test case generation approaches to automatically generate test cases at the unit testing level using various techniques, e.g., random techniques (Pacheco et al., 2007; Pacheco and Ernst, 2007), search-based techniques (Łukasczyk and Fraser, 2022; Fraser and Arcuri, 2011), and deep learning techniques (Watson et al., 2020; Tufano et al., 2020; Alagarsamy et al., 2023). However, these test case generation approaches often take code as input, which does not apply to TDD, as TDD mandates writing test cases before coding. Thus, it remains challenging for developers to follow the TDD practices, i.e., writing unit tests based on a given textual description (i.e., the function description). Therefore, a novel Text-to-Testcase generation approach is required to help developers write unit tests based on a given textual description, which may improve the overall efficiency of automated software testing.

In this paper, we propose PyTester, a Text-to-Testcase generation approach for Python using Deep Reinforcement Learning (Deep RL). Our Deep RL-based model is optimized with Proximal Policy Optimization (PPO) on a reward function that considers three types of feedback,

☆ Editor: Christoph Treude.

* Corresponding author.
E-mail addresses: chakkrit@monash.edu, chakkrit.tantithamthavorn@monash.edu (C. Tantithamthavorn).

https://doi.org/10.1016/j.jss.2025.112381
Received 13 February 2024; Received in revised form 19 January 2025; Accepted 9 February 2025
Available online 17 February 2025
0164-1212/© 2025 The Authors. Published by Elsevier Inc. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

namely, syntax correctness, test executability, and code coverage. We evaluate our approach on four evaluation aspects, i.e., syntax correctness, passing test case, test completeness measured by code coverage, and test effectiveness measured by mutation score. Evaluating the alignment of the test cases to a given textual description is indeed challenging. To address this challenge, we define that a test case is aligned with a given textual description if the test case is executable (i.e., all assertions are passed.) with the ground truth code, assuming that the ground truth code is the correct implementation of the given textual description. Then, we compare our PyTester with five strong large language models (LLMs), namely, CodeT5-large (Le et al., 2022), GPT3.5 (OpenAI, 2023), Copilot (GitHub, 2021), StarCoder (Li et al., 2023), and InCoder (Fried et al., 2022). Through a comprehensive experiment on the APPS benchmark dataset (Hendrycks et al., 2021), we address the following research questions:

RQ1. What is the performance of PyTester for the Text-to-Testcase generation task when compared to the state-of-the-art models?
Of the test cases generated by PyTester, 99% are syntactically correct, 84% are passed (i.e., correct test cases) while achieving a code coverage of 80% and a mutation score of 61%, which outperforms all of the studied LLMs. This finding demonstrates that a smaller language model like PyTester can outperform larger language models when carefully designed. Importantly, after being optimized with Deep RL, PyTester outperforms GPT3.5 for all evaluation metrics, highlighting the effectiveness of our Deep RL framework for Text-to-Testcase generation.

RQ2. How does the choice of the feedback types impact the performance of our PyTester?
The top-3 reward functions for our PyTester are the Syntax+Coverage, Syntax+Executability, and Syntax-only types of feedback. Without considering syntax correctness, model performance decreases by 3%–13%, indicating that syntax correctness must be the minimum consideration when designing a reward function for Deep RL in Text-to-Testcase generation.

RQ3. What types of runtime errors occur in the PyTester-generated test cases?
As low as 15.59% of the PyTester-generated test cases are incorrect. Among the incorrect test cases, PyTester mostly encounters an Assertion Error issue for 10.75%, followed by other types of errors (i.e., IndexError, ValueError, EOFError, SyntaxError).

These results lead us to conclude that the performance of smaller language models like PyTester is not necessarily inferior to that of large language models if carefully designed. Our results confirm that Deep Reinforcement Learning substantially contributes to performance improvements. Deep RL enables smaller language models, with the smallest model parameter size and the fastest inference times by *at least* one order of magnitude, to outperform much larger language models. Furthermore, our findings indicate that incorporating test case characteristics into the reward function significantly enhances performance, resulting in the model generating test cases that are 6% more syntactically correct, 5% more aligned with the description (i.e., passing test case) with 6% higher code coverage and 7% higher mutation score when compared to the GPT3.5 large language model. These findings emphasize the importance of considering domain knowledge and test case characteristics when designing a Deep RL-based Text-to-Testcase generation approach.

Additionally, we conduct further comprehensive evaluations on the HumanEval (Chen et al., 2021) and the MBBP (Austin et al., 2021) benchmark datasets using PyTester in zero-shot learning (i.e., trained only on the APPS (Hendrycks et al., 2021) dataset). We also added Pynguin (Lukasczyk and Fraser, 2022) to the baselines for a Code-to-Testcase comparison. As a result, on the HumanEval dataset, PyTester ranks second place in generating executable test cases with high code coverage, following only GPT-3.5. On the MBPP dataset, PyTester performance is comparable to the 15.5B StarCoder Model. Notably, the input (i.e., a textual description) of each dataset has different formats and details, potentially impacting the results of PyTester that trained on APPS. Nonetheless, the findings indicate that PyTester still performs considerably well even in zero-shot learning comparable to much larger language models.

Finally, we acknowledge the importance of the requirement understanding to generate test cases by software engineers. Thus, the main purpose of PyTester is only to assist the TDD process during the test case generation step—*neither to fully automate the TDD process nor replace human involvement.* Although our PyTester can generate test cases based on a given textual description, it remains essential for software engineers to review and refine these generated test cases to ensure their correctness and completeness.

Novelty & Contributions. The novelty and contributions of this paper are as follows:

- We, *conceptually*, introduce PyTester, a Text-to-Testcase generation approach for a single call function that can automatically generate syntactically correct, executable, complete, and effective test cases while being aligned with a given textual description.
- We, *technically*, formulate the Text-to-Testcase generation task as a deep reinforcement learning problem where the reward function is designed to consider various characteristics of test cases, including syntax correctness, test executability, and code coverage.
- We, *empirically*, demonstrate that on the APPS dataset, our PyTester model outperforms all of the studied state-of-the-art Large Language Models including GPT3.5, Copilot, StarCoder, and InCoder, while being the smallest language model with the fastest inference time.

To support the open science initiatives and increase the verifiability of our study, the replication package of PyTester is available at https://github.com/awsm-research/pytester.

2. Background and motivation

In this section, we describe the background of TDD, discuss existing automated test case generation approaches and a motivating example of ChatGPT for Text-to-Testcase generation.

2.1. Test-Driven Development (TDD)

Test-Driven Development (TDD) (Beck, 2022; Astels, 2003) is a software development methodology that involves writing tests for a piece of code before actually implementing the code itself. Aligned with the Shift-Left Testing principles and Agile manifesto (Beck et al., 2001; Ambler, 2004), TDD is typically carried out in small, incremental steps, with the developer ① *writing a failing test based on textual descriptions, then* ② *writing the minimum amount of code required to make the test pass, and finally* ③ *refactoring the code as needed.* With TDD, developers need to first understand the descriptions and think about the expected behavior of their code, leading to more reliable and maintainable code. Prior studies also found numerous benefits of TDD, including increasing requirements understanding (Mueller and Hagner, 2002), and producing more clean code with a higher number of test cases (Janzen and Saiedian, 2008; Erdogmus et al., 2005). However, the process of writing test cases remains manual, time-consuming, and labor-intensive. This limitation highlights the need for an automated test case generation approach based on textual descriptions, which could potentially accelerate the TDD process and enhance developer productivity.

2.2. Automated test case generation approaches

Various automated test case generation approaches are proposed with different techniques.

Random-based test case generation (Randoop), proposed by Pacheco et al.(Pacheco et al., 2007; Pacheco and Ernst, 2007), aims to automatically generate unit tests for Java classes using feedback-directed random test generation. However, the generated test cases are still suboptimal (e.g., miss edge cases or fail to cover specific scenarios) and non-deterministic (e.g., produce different sets of test cases from different runs).

Search-based test case generation (e.g., Evosuite (Fraser and Arcuri, 2011) for Java, and Pynguin (Lukasczyk and Fraser, 2022) for Python) leverages evolutionary search algorithms to guide the search for suitable test cases to produce test suites that achieve high code coverage. However, prior studies raised concerns that the test cases generated by search-based approaches are often not meaningful (Almasi et al., 2017) and ineffective (Grano et al., 2019; Palomba et al., 2016).

Deep learning-based test case generation (e.g., ATLAS (Watson et al., 2020), Athena (Tufano et al., 2020), and A3Test (Alagarsamy et al., 2023)) formulated the task as a Neural Machine Translation (NMT) problem (Java Method→Test). However, such DL-based approaches aim to generate test cases that are exactly matched with the ground truth test cases, without considering the alternative test cases.

NLP-based test case generation (e.g., Jdoctor (Blasi et al., 2018), and TOGA (Dinella et al., 2022)) applied pattern, lexical, and semantic matching to translate Javadoc comments into test cases (Jdoctor) or a unified transformer-based approach to translate the context of a Java Method into assertion tests (TOGA). However, these approaches are limited to Java programming language.

Limitations. While different test case generation approaches are proposed, none of these approaches are designed for TDD (i.e., taking textual description as input), are not able to generate test cases without code (i.e., mostly they take code as input) and are mostly designed for Java—not Python. Therefore, these limitations highlight the need for a Python automated test case generation approach that takes a description as input and can generate alternative test cases beyond the ones that exactly match the ground truth test cases.

2.3. ChatGPT for text-to-testcase generation: Motivation

ChatGPT (OpenAI, 2023) is a powerful AI conversational large language model (LLM) designed for generating natural text based on a given prompt question in a dialogue style. Recently, researchers found that ChatGPT can perform very well in various software engineering tasks (Hou et al., 2023), including code generation (Chen et al., 2022) and test case generation (Schäfer et al., 2023; Siddiq et al., 2023; Tang et al., 2023; Xie et al., 2023; Yuan et al., 2023). For example, Chen et al. (2022) proposed CodeT, a code generation approach based on a heuristic ranking process using a dual execution agreement of ChatGPT-generated codes and test cases. Schäfer et al. (2023) proposed TestPilot, a ChatGPT-based test case generation approach for JavaScript based on a given input function. Siddiq et al. (2023) investigated the performance of LLMs for test case generation based on a given Java focal method (i.e., methods to be tested). Similarly, many recent work (Tang et al., 2023; Xie et al., 2023; Yuan et al., 2023) also investigated the performance of ChatGPT-based test case generation based on a given Java focal method.

While ChatGPT has been widely used for test case generation, existing work only focuses on taking code as input—*not the natural language descriptions as input*. Therefore, existing test case generation approaches do not support TDD, as TDD mandates writing test cases based on textual descriptions before writing the actual code. In addition, a systematic literature review by Hou et al. (2023) also confirmed that test case generation approaches to support TDD (i.e., LLM-based Text-to-Testcase generation) remain largely unexplored. To address this research gap, we first conduct a preliminary analysis to empirically investigate the performance of ChatGPT for Text-to-Testcase generation.

An Illustrative Example. We use ChatGPT through the OpenAI API by selecting the recommended *GPT-3.5-turbo* model endpoint. Notably, the experiment was run in June 2023. Following Chen et al. (2022), our prompt includes the context (i.e., external information or additional context that can steer the model to better responses) and the instruction (i.e., a specific task or instruction that the model should perform). Fig. 1 (left) presents an example of the input format (i.e., prompt format) where the context refers to the signature of the Python function with a natural language description in the docstring, without the code implementation (Fig. 1 (top right)), while the instruction refers to a natural language comment "*# check the correctness of the '...' function*" followed by an incompleted `assert` statement. We randomly select one programming task (Problem #4479) from the testing set of the APPS dataset (Hendrycks et al., 2021) as an illustrative example. Note that a comprehensive evaluation of ChatGPT is provided in Section 5 where we found that our PyTester performs better. For the given Problem #4479, we reformat the data structure into a triplet of <*function signature* + *description text*, *code implementation*, *test cases*>. Then, the *function signature* + *description text* is input into ChatGPT to generate test cases, while *code implementation* is only used to evaluate if the generated test cases can be run against the ground truth code.

Observation-1: ChatGPT can generate syntactically correct test cases, yet some of the generated test cases cannot be executed (i.e., failed test cases). In reality, when using ChatGPT to generate test cases based on a given description, it becomes challenging to verify if the generated test cases are correct and aligned with the given description or not. To address this challenge, assuming that the ground truth code is the correct implementation, we execute the test cases against the ground truth code to verify if the ChatGPT-generated test cases are executable or not. Fig. 1 (bottom right) shows that four test cases generated by ChatGPT are syntactically correct, but the fourth test case is not executable. In particular, the fourth test case is generated with the test input of `'([-2,9,-8,8,7], 3)'`. Thus, the expected output should be `'30'`, but incorrectly generated as `'32'`. Therefore, this test case will raise an *AssertionError*. Such syntactically correct, yet non-executable test cases indicate that the test case is still not aligned with the given descriptions. This observation demonstrates that ChatGPT is able to generate syntactically correct test cases, yet fails to generate executable ones (i.e., failed test cases).

Observation-2: When ChatGPT generates non-executable test cases, it becomes challenging to evaluate the completeness and effectiveness of the test cases. Intuitively, effective test cases should exercise all lines of code in the program (i.e., achieving high code coverage), and be able to discover bugs/defects when the program is slightly changed (i.e., the same set of test cases is able to catch the introduced bugs). Code coverage is widely used to measure test completeness (Pacheco and Ernst, 2007; Fraser and Arcuri, 2011; Schäfer et al., 2023; Siddiq et al., 2023; Tufano et al., 2020; Alagarsamy et al., 2023; Tang et al., 2023). Thus, a higher code coverage percentage often increases the confidence that the software behaves as intended. However, studies (Hemmati, 2015; Cai and Lyu, 2005) argue that code coverage only measures the extent to which the test cases exercise the lines of code in the program, but not the effectiveness of the test cases. Therefore, mutation testing is proposed to evaluate the effectiveness of the test cases (Cai and Lyu, 2005; Madeyski, 2010; Jia and Harman, 2010; Papadakis et al., 2019). Mutation testing aims to introduce intentional faults (mutations) into the code and check if the existing test cases can detect these faults (mutants)—*referring to the fault detection capability of the test cases*. Thus, the mutation score is measured as a percentage based on the number of mutants that are killed (i.e. detected by the test cases) versus the total number of mutants introduced. Nonetheless, despite the benefits of code coverage and the extensive research on mutation testing in the literature, another

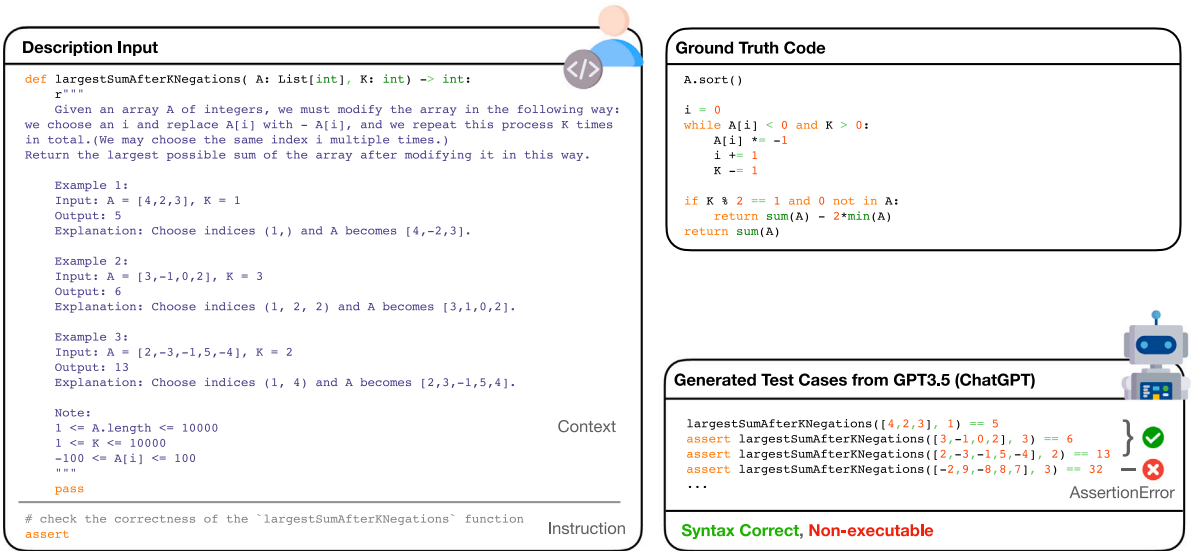

Fig. 1. An example of test cases generated by the OpenAI's GPT3.5 model based on a given description input. Syntax correctness is measured by an AST parser, while the test executability is measured by running the test cases against the ground truth code.

crucial factor is the test case executability. When ChatGPT generates non-executable test cases, such critical metrics (e.g., code coverage and mutation score) are unable to be assessed. Therefore, the models should consider these domain knowledge metrics during training – which is largely overlooked in the existing ChatGPT-based approaches. For instance, although (Chen et al., 2022) proposed a method to re-rank generated code and test cases based on execution feedback, it does not take into account the test case characteristic.

Summary. TDD offers numerous benefits to software development, yet is time-consuming and expensive. Existing test case generation approaches do not yet support TDD practices (i.e., code, *not textual description*, is treated as input). Our motivating analysis shows that ChatGPT, one of the most powerful language models, still generates non-executable, incomplete, and ineffective test cases, hindering the adoption of ChatGPT and other LLMs for Text-to-Testcase generation in practice. To support TDD practices, this observation highlights the need for a novel automated test case generation approach that can generate syntactically correct, executable, complete, and effective test cases.

3. PyTester

Aim. We introduce PyTester, a Text-to-Testcase generation approach that aims to automatically generate syntactically correct, executable, complete, and effective test cases based on a given natural language description. We formulate the Text-to-Testcase generation task as a Reinforcement Learning (RL) problem due to the following reasons:

- **To increase models' ability to generate alternative test cases.** Traditionally, supervised finetuning approaches are trained to generate test cases that are textually and exactly matched with the ground truth test cases. Thus, these models can only generate test cases based on what the model had seen in the training data, limiting the model's ability to generate test cases outside of the domain (i.e., *out-of-distribution (OOD) problems* (Shen et al., 2021)). In fact, the ground truth test cases may not be representative of all of the possible test cases. Thus, alternative test cases with different test inputs that still satisfy the given description should also be considered as valid.

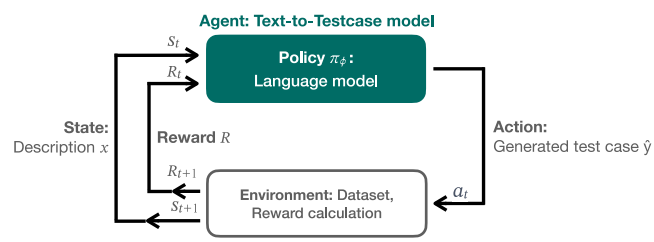

Fig. 2. The formulation of Text-to-Testcase generation as a Reinforcement Learning problem.

- **To incorporate test case characteristics into the feedback loop.** Traditionally, the generated test cases are evaluated only on their similarity with the ground truth test cases. However, in real-world practices, developers normally design test cases by ensuring test cases are syntactically correct, executable, and complete. Such test case characteristics are critically important, yet remain largely disregarded by the existing DL-based test case generation approaches.

PyTester leverages a Deep Reinforcement Learning (Deep RL) framework, which is a combination of deep learning and reinforcement learning principles. In the context of the Text-to-Testcase generation, the goal of RL is to enable the agent (i.e. the Text-to-Testcase generation model) and its policy (the brain of the agent) to learn by interacting with the environment (through trial and error) and receiving rewards (negative or positive) as feedback for performing actions (i.e. generating test cases). This formulation enables PyTester to (1) learn autonomously through interaction with their environment without ground truth test cases, allowing the models to generate alternative test cases and (2) learn how to generate syntactically correct, executable, complete, and effective test cases while being aligned with a given description through a carefully-crafted reward function rather than a basic loss function used in traditional finetuning models.

3.1. Our reinforcement learning framework for text-to-testcase generation

We represent the reinforcement learning problem as a Markov Decision Process (MDP) which decomposes scenarios as a series of states, connected by actions and associated with a specific reward. Following Fig. 2, we define and explain each of the components below.

Language Model: Given the natural language *description* x and a *testcase* y, the Text-to-Testcase generation language model is defined as $P(y|x) = \prod_{i=0}^{m} p_\theta(y_i|x, y_{0:i-1})$

Dataset: We define a training dataset D of size M, which contains triplets of <*description*, *code*, *test cases*> as $D = \{(x^i, c^i, y^i)\}_{i=1}^{i=M}$. The description x consists of a signature of the Python function, the natural language description included in the docstring, the hidden code implementation (`pass`), the instruction prompt as a natural language comment "`# check the correctness of the '...' function`", and an incompleted assert statement "`assert`". On the other hand, the ground truth test case y consists of an assertion statement.

Agent/Policy: Our policy π_ϕ is defined as a language model p_θ, training from dataset D. The policy outputs the probability of all vocabularies conditioned on the current state: $\pi_\phi(\cdot|s_t)$.

State: A state $s_t = (x, \hat{y}_{0:t-1})$ is defined as a concatenation of the **textual description** x and the **test case** tokens $\hat{y}_{0:t-1}$ generated so far.

Action: An action $a_t = \hat{y}_t \sim \pi_\phi(\cdot|s_t)$ is a token sampled from the policy. The action is then used as a continuation of the current state by the **transition function**, T, to produce the next state, s_{t+1}. Specifically, the next state is the concatenation of the action and the current state: $s_{t+1} = T(s_t, a_t) = (x, \hat{y}_t)$, where $\hat{y}_t = a_t \oplus \hat{y}_{0:t-1}$.

With the Markov Decision Process (MDP), our PyTester agent is learned as an episodic task. Generally, an episode is a sequence of interactions between the agent and environment starting from an initial state s_0 and ending at a terminal state where the agent will receive the reward and update the policy. At the beginning of each episode, the *initial state* will start from a description x sampled from the Dataset D, denoted as $s_0 = x \sim D_x$. Then, our policy π_ϕ takes a description x as input to generate a test case $\hat{y}$ that is syntactically correct, executable, and complete. Technically, the policy π_ϕ will sequentially generate a sequence of tokens $\hat{y}_i$ for the test case $\hat{y}$ until reaching the *end of the trajectory* (i.e., after generating the `EOS` token or reaching the maximum generation length). Then, the generated test cases $\hat{y}$ are input into the interactive environment to receive a reward via a Reward Calculation function $R(x, \hat{y})$, which returns a cumulative **reward point**. The reward will be provided if the generated test cases are syntactically correct, executable against the ground truth code (i.e., aligned with the descriptions), and complete (i.e., achieving high code coverage). On the other hand, the penalty will be provided if the generated test cases are syntactically incorrect and non-executable (e.g., runtime errors like incorrect method names and assertion errors like wrong test inputs). Finally, the policy π_ϕ is learned over time in order to maximize a reward function written as:

$$\mathcal{J}(\pi_\phi) = \mathbb{E}_{x \sim D_x, \hat{y} \sim \pi_\phi(\cdot|x)} R(x, \hat{y}) \tag{1}$$

Overview. Fig. 3 presents an overview of our PyTester framework. Our framework consists of two steps: policy training and policy optimization. In the *policy training* step, we aim to build the agent (i.e., the Text-to-Testcase Generation model) where its policy interacts with the environment (i.e., the input dataset) and learns to make decisions (i.e., generate test cases) over time. However, currently, the agent is still suboptimal as it is trained to generate the same test cases as the ground truths in a supervised manner. Thus, the *policy optimization* step aims to further optimize the agent via a reward function through Deep Reinforcement Learning. To do so, given an environment (i.e., a triplet of <*description*, *code*, *test cases*> in the training dataset), the agent will take the description as input into the model in order to perform an action (i.e., generate test cases), where the test cases will be evaluated via a reward function. Our reward function is designed to incorporate three aspects of test case characteristics, namely syntax correctness, test executability, and test completeness. The agent is optimized with Proximal Policy Optimization (PPO) (Schulman et al., 2017) using our reward function, and stabilized with KL-divergence (Kullback and Leibler, 1951). Below, we describe the technical details of policy training, policy optimization, and our reward function.

3.2. Policy training

The ultimate goal of the policy training is to build a language model with some preliminary knowledge of the relationship between the description and its associated test cases. Therefore, we adopt the *default CodeT5-large* as our language model (Le et al., 2022; Raffel et al., 2020). To ensure that the model performs well on a target task (i.e., Text-to-Testcase generation), we finetune our language model with the training dataset. Given a training dataset $D = \{(x^i, c^i, y^i)\}_{i=1}^{i=M}$, we finetune the language model to learn the mapping between the description x and its corresponding test cases y (not the ground truth code c) in order to minimize the multi-class cross-entropy loss $\mathcal{L}_{\text{FT}} = -\mathbb{E}_{\substack{(x,y)\sim D \\ n\in\{0,\ldots,N\}}} \log p_\theta(y_n|x, y_{0:n-1})$, where N is the maximum length of the test cases. Then, the loss is optimized using an Adam optimizer (Kingma and Ba, 2014) (the hyperparameters settings are provided in Table 2).

3.3. Policy optimization

Next, we further update the PyTester policy with a reward function through reinforcement learning. At the initial state, we initiate two finetuned language models from the policy training step: a language model policy π_ϕ as our active model (i.e., to-be trained model) and a language model p_θ as a reference model (i.e., the frozen model that is not updated during the policy optimization step). We optimize our policy using PPO and KL-divergence as follows.

Proximal Policy Optimization (PPO) (Schulman et al., 2017). We use PPO to improve the training stability of the policy optimization process. Formally, PPO optimizes the following surrogate objective:

$$\mathcal{J}_{\text{surr}}(\pi_\phi) = \min(rA^{\pi_{\phi_{\text{old}}}}(x, \hat{y}), \text{clip}(r, 1-\epsilon, 1+\epsilon)A^{\pi_{\phi_{\text{old}}}}(x, \hat{y})), \tag{2}$$

$$\text{where} \quad r = \frac{\pi_\phi(\hat{y}|x)}{\pi_{\phi_{\text{old}}}(\hat{y}|x)}, \tag{3}$$

in which ϵ is a small positive scalar indicating how much the policy can be updated and $\pi_{\phi_{\text{old}}}$ is the policy that collects the data. $A^\pi(x, \hat{y})$ is the advantage function of a policy π given a state–action (i.e., input text description - output test cases) pair $(x, \hat{y})$, and is computed by the Generalized Advantage Estimator (GAE) (Schulman et al., 2015).

KL-divergence (Kullback and Leibler, 1951). Following best practices (Ouyang et al., 2022; Stiennon et al., 2020; Ziegler et al., 2019), we utilize the Kullback–Leibler (KL) divergence regularization (Kullback and Leibler, 1951) to stabilize the training process. KL-divergence is a penalty to prevent our active model π_ϕ from exploring too differently from the reference model p_θ (i.e., the finetuned model). Concretely, we define the KL-divergence as $KL_{penalty}$ and the reward function R as

$$R(x, \hat{y}) = R_{\text{task}} - KL_{\text{penalty}} = R_{\text{task}} - \beta \log \frac{\pi_\phi(\hat{y}|x)}{p_\theta(\hat{y}|x)}, \tag{4}$$

where R_{task} is the summation of the rewards and $\beta > 0$ is a hyperparameter controlling the strength of the regularization.

3.4. Reward functions

Once the test cases $\hat{y}$ are generated from the model, the generated test cases are evaluated against our reward function. Our reward function aims to consider three types of feedback, namely, syntax correctness, test executability, and test completeness. In general, each reward function will provide a reward point for positive feedback or a penalty point for negative feedback. Our policy is optimized based on the following reward function:

$$R_{task} = R_{syntax} + R_{exec} + R_{cc} - P_{syntax} - P_{exec} - P_{assert} \tag{5}$$

Below, we define the three types of feedback as follows:

Syntax Correctness Feedback concerns the syntactically correctness of the generated test cases $\hat{y}$. To determine if the generated test

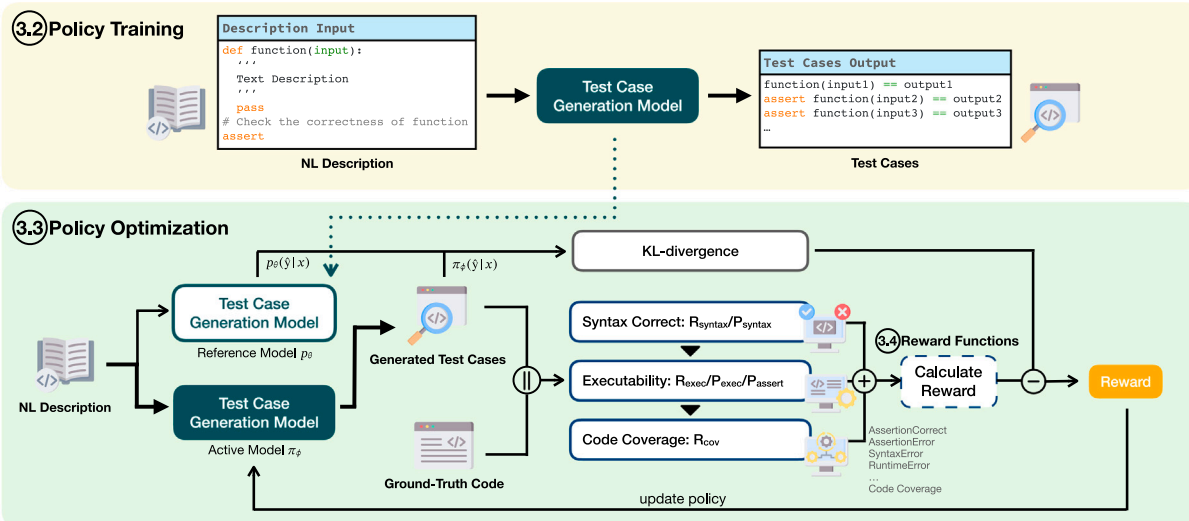

Fig. 3. An overview of our Deep Reinforcement Learning framework for the Text-to-Testcase Generation task (called PyTester). ‖ denotes a concatenation, + denotes an addition, and − denotes a subtraction.

cases are syntactically correct, we use the Python `ast` library[1] to parse the generated test cases $\hat{y}$. We define Condition $\mathbb{1}_{syntax(\hat{y})} = 1$ if $\hat{y}$ is syntactically correct (i.e., successfully parsed by AST). Then, the feedback is provided to the policy based on the following reward/penalty function given reward/penalty point (r_{syntax}, p_{syntax}):

$$R_{syntax} = \mathbb{1}_{syntax} r_{syntax}, \tag{6}$$

$$P_{syntax} = (1 - \mathbb{1}_{syntax}) p_{syntax}, \tag{7}$$

Test Executability Feedback concerns the functional correctness of the generated test cases $\hat{y}$ given the ground truth code c. To determine if the generated test cases are executable and functional correct to the ground truth code, we use the built-in function `exec`.[2] Specifically, we decompose the executability feedback into two conditions: Execution Condition ($\mathbb{1}_{exec(c,\hat{y})}$) and Assertion Condition ($\mathbb{1}_{assert(c,\hat{y})}$). We define Condition $\mathbb{1}_{exec(c,\hat{y})} = 1$ if the execution of $(c, \hat{y})$ does not raise a `RuntimeError` (i.e., the test cases are executable) and Condition $\mathbb{1}_{assert(c,\hat{y})} = 1$ if the execution does not raise an `AssertionError` (i.e., execution passes all the generated assertion test cases). Then, the feedback is provided to the policy based on the following reward/penalty function given reward/penalty point $(r_{exec}, p_{exec}, p_{assert})$:

$$R_{exec} = \mathbb{1}_{syntax} \mathbb{1}_{exec} \mathbb{1}_{assert} r_{exec}, \tag{8}$$

$$P_{exec} = \mathbb{1}_{syntax} (1 - \mathbb{1}_{exec}) p_{exec}, \tag{9}$$

$$P_{assert} = \mathbb{1}_{syntax} \mathbb{1}_{exec} (1 - \mathbb{1}_{assert}) p_{assert} \tag{10}$$

Code Coverage Feedback concerns the percentage of the line of code (LOC) in ground truth code c tested by generated test cases $\hat{y}$. To determine the percentage, we use `coverage.py`[3] to calculate the code coverage, denoted by $CodeCoverage(c, \hat{y})$, which has a value range of $[0, 100]$. Then, the feedback is provided to the policy based on the following reward function given a normalized $CodeCoverage(c, \hat{y})$ as reward point (r_{cov}):

$$R_{cov} = \mathbb{1}_{syntax} \mathbb{1}_{exec} \mathbb{1}_{assert} r_{cov} \tag{11}$$

[1] https://docs.python.org/3/library/ast.html
[2] https://docs.python.org/3/library/functions.html#exec
[3] https://coverage.readthedocs.io/en/7.2.7/

4. Experimental setup

In this section, we present the motivation of our three research questions and describe our experimental setup for evaluating our PyTester approach and the baselines.

RQ1. What is the performance of PyTester for the Text-to-Testcase generation task when compared to the state-of-the-art models? *Motivation.* Prior studies found that large language models (e.g., GPT3.5) tend to outperform smaller language models for various SE tasks (OpenAI, 2023; Rozière et al., 2023). However, little is known about whether our PyTester which is based on a smaller language model like CodeT5 will outperform large language models.

RQ2. How does the choice of the feedback types impact the performance of our PyTester? *Motivation.* The reward function plays an important role in providing a positive/negative signal to the agent if the action satisfies the goal. There exist multiple variations of the reward functions (e.g., considering either one or multiple types of feedback). Yet, little is known about which types of feedback are most important.

RQ3. What types of runtime errors occur in the PyTester-generated test cases? *Motivation.* Our PyTester approach may generate both correct and incorrect test cases. Thus, a manual analysis of the incorrectly generated test cases may offer opportunities for future researchers to consider improving the performance of our approach. Yet, little is known about what are the most common types of errors that are found in the incorrectly generated test cases by PyTester.

4.1. Studied dataset

The APPS benchmark dataset (Hendrycks et al., 2021) is used to evaluate our approach and compare it with the baseline approaches. The APPS benchmark dataset consists of 10,000 programming tasks with 131,777 associated test cases collected from Codewars, AtCoder, Kattis, and Codeforces. The dataset consists of three difficulty levels of the programming tasks, including the introductory level, the interview level, and the coding competition level. Each of the programming tasks consists of triplets of <*a description, an associated code, and associated test cases*>. Similar to prior studies (Hendrycks et al., 2021), we use the same dataset splitting of 5000 samples for training and 5000 samples for testing.

Notably, in this study, only the APPS benchmark dataset is suitable for training our Text-to-Testcase generation task, as our PyTester model requires a large enough size of the dataset for policy training. Nevertheless, to broaden our evaluation, we provide additional analysis of PyTester in a zero-shot setting using **MBPP** (Austin et al., 2021) and **HumanEval** (Chen et al., 2021) datasets. More details can be found in the discussion section (see Section 6).

4.2. Data preprocessing

The quality of the dataset may have an impact on the performance of PyTester. Thus, we carefully craft the dataset for model training through our data preprocessing as follows.

Data Reformatting. After manual analysis, we found that, in the APPS dataset, different programming tasks have different approaches to test case execution. In particular, some programming tasks are tested with an assert statement format (`assert add(1,1) == 2`), while some are tested with a standard input–output format (`python add.py 1 1 2`). Thus, it becomes challenging for a DL model to automatically handle such inconsistency. In particular, for such programming tasks with the standard input–output format, the method name does not exist and the assert statements are missing. To address this challenge, for programming tasks written in a standard input–output format, we transform them by giving a function name to the programming task, then adding a test method (e.g., `def test_main()`) to be called by assert statements (see an example in Fig. 4). Finally, all of the programming tasks in the dataset will have a consistent format of assertion calls.

Data Filtering. In addition, we found that not all samples in the dataset are of high quality. Common issues include missing test cases, missing code solutions, and non-executable test cases. Since the goal of PyTester is to generate test cases that are aligned with the natural language description, our selection criteria are defined as the test cases that must exist and be executable against the code solution. After the data filtering step, our training dataset consists of 3401 samples and our testing dataset consists of 3259 samples. During the model training step, we use 10% of the training dataset for validation, while the rest of the training dataset is used for model training.

4.3. Evaluation metrics

Since the goal of PyTester is to generate syntactically correct, executable, complete, and effective test cases that are aligned with the description, our evaluation will focus on the following four aspects: syntax correctness, passing test case, code coverage, and mutation score.

- **Syntax Correctness** measures the percentage of the generated test cases that are syntactically correct. The syntax correctness is defined by the `ast` Python library.[4] The given test case is considered syntactically correct if the AST can be successfully run.
- **Passing Test Case** measures the percentage of the passing generated test cases to the ground truth code. Since the ground truth code is the correct implementation of the natural language description, any test cases that can be successfully executed against the ground truth code are considered passed (i.e., call the correct functions to be tested with the right test inputs and the expected outputs). Thus, we measure the passing test case by executing the concatenation of the ground truth code with the generated test cases using the `exec` built-in function.[5]

Table 1
The operators used in our mutation testing.

Mutation Testing Operator
AOD - arithmetic operator deletion
AOR - arithmetic operator replacement
ASR - assignment operator replacement
BCR - break continue replacement
COD - conditional operator deletion
COI - conditional operator insertion
CRP - constant replacement
DDL - decorator deletion
EHD - exception handler deletion
EXS - exception swallowing
IHD - hiding variable deletion
IOD - overriding method deletion
IOP - overridden method calling position change
LCR - logical connector replacement
LOD - logical operator deletion
LOR - logical operator replacement
ROR - relational operator replacement
SCD - super calling deletion
SCI - super calling insert
SIR - slice index remove

- **Code Coverage** measures the completeness of the generated test cases for a given natural language description. Ideally, in addition to the passing test case, the test cases must be complete (i.e. executing all lines of the ground truth code solution). Thus, we measure the line-level code coverage of the generated test cases using the `coverage.py` Python library.[6]
- **Mutation Score** measures the effectiveness of the generated test cases. High-quality test cases must be able to detect bugs if the code is slightly changed. Mutation testing aims to introduce intentional faults (mutations) into the code and check if the existing test cases can detect these faults (mutations). Then, the mutation score is measured as a percentage based on the number of mutations that were killed (detected) versus the total number of mutations introduced. We use the `MutPy` Python library[7] to measure a mutation score with a timeout of 30 s per sample. We set the configuration to default with `PyTest`[8] as a test runner. The mutation operators are listed in Table 1.

Different from prior studies that aim to generate test cases that are textually and exactly matched with the ground truth test cases, we neither use such an exact match nor a code similarity measure (e.g., a BLEU score) in this study.

4.4. Evaluation setting

Since the number of test cases generated from each model varies, we provide 3 evaluation settings: Raw, *n*-test, and Filtered by Syntax Correctness, to ensure a fair comparison.

- **Raw** evaluates the generated test cases directly from the model without any post-processing. In this setting, the test cases are assessed as a whole set. If any single test case fails, the entire output is considered incorrect.
- ***n*-test** evaluates the generated test cases by limiting the size to n test cases. We use $n = 3$ (3-test) and $n = 5$ (5-test).
- **Filtered by Syntax Correctness** evaluates the generated test cases after filtering out the incorrect syntax of the test cases. First, we split multiple assertions into individual tests, then exclude the syntactically incorrect test cases before evaluation. Notably, the test case will be considered incorrect, if the generated text is an empty string or all of the test cases are filtered out.

[4] https://docs.python.org/3/library/ast.html
[5] https://docs.python.org/3/library/functions.html#exec
[6] https://coverage.readthedocs.io/en/7.2.7/
[7] https://pypi.org/project/MutPy/
[8] https://docs.pytest.org/en/latest/

4.5. Baselines

Since the existing test case generation approaches (discussed in Section 2) are not designed for TDD (i.e. do not take a description as input), it becomes challenging to fairly compare our PyTester with the existing ones. Since prior studies demonstrated that Large Language Models (LLMs) could be a viable solution for various SE tasks, we consider the following five baseline models. Note that if available, we use beam search with a size of 5 as the decoding method. Otherwise, we conduct the experiments five times per data point and report the mean and standard deviation.

- **CodeT5-large** (770M parameters) (Le et al., 2022) is a transformer language model pre-trained on six programming languages of CodeSearchNet (Husain et al., 2019) with the masked span prediction task (Wang et al., 2021).
- **InCoder** (6.7B parameters) (Fried et al., 2022) is a transformer-based LLM pre-trained on a total of 159 GB public code with permissive open source licenses from GitHub and GitLab, in which 52 GB are Python, and a total of 57 GB of text content are from StackOverflow.
- **StarCoder** (15.5B parameters) (Li et al., 2023) is a transformer-based LLM pre-trained on 80 different programming languages and natural languages such as GitHub issues, commits, and notebooks. We use the finetuned version which is trained with 35B Python tokens.
- **GPT-3.5** (>175B parameters[9]) (OpenAI, 2023) is one of the most powerful state-of-the-art LLMs developed by OpenAI, which is a transformer-based LLM pre-trained with the next token prediction task on the internet and the third-party licensed data, then finetuned using Reinforcement Learning from Human Feedback (RLHF) (Christiano et al., 2017). We chose the recommended *gpt-3.5-turbo* endpoint, accessed in June 2023.
- **Copilot** (>12B parameters[10]) (GitHub, 2021) is another one of the most powerful state-of-the-art LLMs from GitHub. It was powered by the OpenAI Codex Model and introduced as an "AI pair programmer" in Visual Studio Code. We include Copilot in this study to reflect the performance of a tool in a realistic coding scenario. Access to Copilot was obtained through the following repository.[11]
- **Pynguin** (Lukasczyk and Fraser, 2022) is a search-based test case generation approach that produces test cases given a source code. We include Pynguin in this study to reflect the performance of a traditional Code-to-Testcase tool. Notably, due to a technical issue with IO functions, Pynguin is incompatible with the APPS dataset. Nonetheless, we include Pynguin in the discussion section (see Section 6), where we assess our PyTester in the zero-shot learning on the MBPP and HumanEval datasets.

4.6. Model experimentation

The performance of our PyTester model heavily relies on the design of our reward function. Thus, considering three different types of feedback, we perform an experiment with the following seven variations:

- Syntax-only (p_{syntax}=-2, r_{syntax}=2)
- Executability-only (p_{exec}=-2, p_{assert}=-0.3, r_{exec}=2)
- Coverage-only (p_{exec}=-2, $r_{cov} = norm([0,2])$)
- Syntax+Executability (p_{syntax}=-2, p_{exec}=-1, p_{assert}=-0.3, r_{exec}=2)
- Syntax+Coverage (p_{syntax}=-2, $r_{cov} = norm([0,2])$)
- Executability+Coverage (p_{exec}=-2, p_{assert}=-0.3, r_{exec}=2, $r_{cov} = norm([0,2])$)
- Syntax+Executability+Coverage (p_{syntax}=-2, p_{exec}=-1, p_{assert}=-0.3, r_{exec}=2, $r_{cov} = norm([0,2])$)

Table 2
The hyperparameter settings of PyTester during the policy training step.

Hyperparameter	Value
Learning rate	2e−5
Warmup steps	1000
Weight decay	0.01
Batch size	2
Gradient accumulation steps	4
Learning rate scheduler type	inverse_sqrt
Epoch	20

Table 3
The hyperparameter settings of PyTester during the policy optimization step.

Hyperparameter	Value
Learning rate	1e−5
KL coefficient	0.2
VF coefficient	0.01
Clip-range	0.05
Clip-range value	0.2
Batch size	8
Mini batch size	4
Gradient accumulation steps	4
Epoch	1

where r is the reward point, and p is a penalty point, or 0 if not specified. For example, when considering 'Syntax-only' as a reward function, a reward point of 2 is given if the generated test cases are syntactically correct, otherwise, a penalty point of −2 is given.

We run our experiment on two NVIDIA GeForce RTX 3090 GPUs with 24 GB vRAM, an Intel(R) Core(TM) i9-9980XE CPU @ 3.00 GHz with 36 core processors, and 64G RAM. The hyperparameter settings for the policy training and the policy optimization are reported in Table 2 and Table 3, respectively.

In total, we experimented with 7 variations. During the training phase, we set the model to learn to generate multiple assert statements (without the length limit). During the inference phase, the test case generation for each model uses a beam search of 5 and each model is evaluated using the following measure, called TestQuality:

$$\text{TestQuality} = \frac{(2 \times CodeCoverage \times MutationScore)}{(CodeCoverage + MutationScore)} \quad (12)$$

Ideally, high-quality test cases must execute all lines of code in the function (i.e., measured via code coverage), while being able to detect bugs if the code is slightly changed (i.e., measured via mutation score). We define the Test Quality score as a harmonic mean of the code coverage and the mutation score. After the comprehensive experiment, we find that **the PyTester model performs best when it is trained to generate multiple assert statements with a reward function considering both Syntax and Coverage feedback,** which we used as a reference setting for our PyTester model for the remainder of the paper.

5. Experimental results

(RQ1) what is the performance of* PYTESTER *for the text-to-testcase generation task when compared to the state-of-the-art models?

Results. **Of the test cases generated by PyTester, 99% are syntactically correct, 84% are passed while achieving a code coverage of 80% and a mutation score of 61%, which outperforms all of**

[9] GPT-3.5 model size is not reported publicly. The previous version (GPT-3 model (Brown et al., 2020)) has a model parameter size of 175B.

[10] Copilot model size is not reported publicly. The previous version (Codex model (Chen et al., 2021)) has a model parameter size of 12B.

[11] https://github.com/B00TK1D/copilot-api/tree/main

Table 4
(RQ1) The performance of PyTester when compared to state-of-the-art LLMs. LLMs without beam search are run 5 times and reported as Mean (SD).

Evaluation setting	Model	Model parameters	Inference time	Syntax correctness	Passing test case	Code coverage	Mutation score
Raw	**PyTester**	770M	<1 h	**99.42%**	**84.41%**	**80.98%**	**61.45%**
	Finetuned CodeT5-large	770M	<1 h	65.08%	35.56%	34.44%	26.94%
	InCoder	6B	250 h	4.63%	0.03%	0.03%	0.03%
	StarCoder	15.5B	40 h	96.78%	71.56%	69.07%	54.32%
	GPT3.5	>175B	12 h	93.59% (0.79%)	79.21% (0.67%)	75.10% (0.68%)	54.32% (0.58%)
	Copilot	>12B	6 h	59.40% (0.87%)	48.73% (0.65%)	47.11% (0.60%)	33.82% (0.62%)
3-test	**PyTester**	770M	<1 h	**99.42%**	**84.44%**	**80.92%**	**60.36%**
	Finetuned CodeT5-large	770M	<1 h	96.01%	53.57%	51.58%	40.23%
	InCoder	6B	250 h	19.24%	0.12%	0.11%	0.07%
	StarCoder	15.5B	40 h	99.29%	72.63%	70.02%	52.59%
	GPT3.5	>175B	12 h	93.73% (0.79%)	80.27% (0.78%)	76.07% (0.77%)	54.53% (0.60%)
	Copilot	>12B	6 h	94.42% (0.33%)	72.47% (0.22%)	70.00% (0.25%)	53.18% (0.29%)
5-test	**PyTester**	770M	<1 h	**99.42%**	**84.41%**	**80.91%**	**60.41%**
	Finetuned CodeT5-large	770M	<1 h	91.53%	38.75%	37.50%	29.42%
	InCoder	6B	250 h	12.83%	0.03%	0.03%	0.03%
	StarCoder	15.5B	40 h	98.96%	71.86%	69.34%	54.38%
	GPT3.5	>175B	12 h	93.72% (0.79%)	79.36% (0.69%)	75.23% (0.68%)	54.07% (0.58%)
	Copilot	>12B	6 h	90.72% (0.53%)	61.15% (0.54%)	59.12% (0.55%)	41.08% (0.66%)
Filtered by Syntax Correctness	**PyTester**	770M	<1 h	99.63%	**84.50%**	**81.06%**	**60.54%**
	Finetuned CodeT5-large	770M	<1 h	98.93%	38.39%	36.90%	24.86%
	InCoder	6B	250 h	1.90%	0.12%	0.11%	0.05%
	StarCoder	15.5B	40 h	**99.97%**	71.95%	69.25%	47.13%
	GPT3.5	>175B	12 h	94.55% (0.70%)	79.79% (0.72%)	75.68% (0.71%)	54.76% (0.59%)
	Copilot	>12B	6 h	99.52% (0.03%)	59.59% (0.95%)	57.58% (0.93%)	39.56% (0.83%)

the studied LLMs. Table 4 presents the performance of PyTester when compared to the state-of-the-art LLMs according to the four evaluation metrics. When comparing PyTester with GPT3.5, our PyTester model is able to generate test cases that are 6% more syntactically correct, 5% more passing, 6% more complete, and 7% more effective in the shortest amount of inference time due to the smallest size of the model. This finding indicates that a small language model like PyTester which is based on CodeT5 does not necessarily perform worse than large language models if they are carefully designed. Instead, by incorporating the domain knowledge of software testing into the architecture of PyTester (i.e., the consideration of test case characteristics in the reward function that is largely ignored by existing LLM models), PyTester is able to perform better than large language models, with a smaller size of the model parameters and a shorter amount of inference time. This highlights the significant novelty of our approach contributing to the research area of Text-to-Testcase generation.

Our proposed DeepRL framework can improve the passing test case percentage by 49%, the code coverage by 47%, and the mutation score by 32% when compared with the CodeT5 base model. Table 4 shows that, without the deep reinforcement learning, the CodeT5 base model performs worse than GPT3.5, which is aligned with the common intuition and prior studies (OpenAI, 2023; Guo et al., 2023). On the other hand, when adding deep reinforcement learning to the CodeT5 base model, our experiment confirms that the PyTester model performs better than the GPT3.5 for all evaluation metrics. This finding highlights the importance of our proposed deep reinforcement learning framework for Text-to-Testcase generation.

Fig. 4 presents an illustrative example of the description input and the test cases by PyTester and the baselines. This example depicts a programming task (Problem ID #569) in the APPS dataset. We observe that PyTester can generate test cases that are syntactically correct and align with the description while achieving high code coverage and high mutation score. On the other hand,

- GPT3.5, Copilot and StarCoder can generate syntactically correct test cases, but some are non-executable, raising an `AssertionError`. We observed that the non-executable test cases have to do with incorrect test inputs and incorrect expected outputs. For example, StarCoder generates a test input with a missing of the first value (e.g., "2\n"), while Copilot and GPT3.5 generate the expected outputs of the test case with an additional "\n". Such incorrectly generated test cases could be due to the lack of consideration of the passing test case during the learning process of the large language models. In contrast, the consideration of syntax correctness and code coverage in the reward function of our PyTester model improves the generated test cases to align to the given description (i.e., executable) with high coverage.
- InCoder often generates incorrect test cases (syntactically incorrect, non-executable, low code coverage, and low mutation score). Like in this example, some issues include the missing of an `assert` call, the incorrect test inputs/outputs, and the incompleted generation. Such incompleted generation happens since InCoder does not generate the terminal tokens (i.e., the EOS token), resulting in a longer inference time and incompleted assert statements.
- CodeT5-large often generates repetitive assert statements. For example, this assert statement "`assert test_main('2\nba') == '1'`" is repeatedly generated two times, while the last assert statement is also incomplete. Such repetitive generation has to do with the traditional learning objective aiming to generate test cases that are exactly matched with the ground truth test cases. Differently, PyTester addresses this challenge by considering the test case characteristics during the test case generation. Rather than evaluating the test cases with a traditional text similarity measure, we instead consider the test case characteristics via a reward function.

(RQ2) how does the choice of the feedback types impact the performance of our* PYTESTER*?

Results. **The top-3 reward functions for PyTester are the Syntax+Coverage, Syntax+Executability, and Syntax-only types of feedback.** Table 5 presents the performance of our PyTester for the seven variations of the reward functions. We find that the Syntax+Coverage, Syntax+Executability, and Syntax-only types of the reward function achieve a comparative TestQualityScore of 69.87%, 69.51%, and 69.00%, respectively. On the other hand, without considering the syntax correctness feedback (for the remaining reward function types), the TestQualityScore of PyTester is decreased by 3%–13%. This finding highlights the importance of syntax correctness in the reward

Inference input

Input text

```
import sys
import io                                   # Header to reformat stdio to function value
def test_main(input_string):
    sys.stdin = io.StringIO(input_string)
    captured_output = io.StringIO()
    sys.stdout = captured_output
    main()
    sys.stdin = sys.__stdin__
    sys.stdout = sys.__stdout__
    actual_output = captured_output.getvalue().strip()
    return actual_output
```

```
def main():                                 # Description text
    r"""
    A wise man told Kerem "Different is good" once, so Kerem wants all thing
s in his life to be different.

    Kerem recently got a string s consisting of lowercase English letters. S
ince Kerem likes it when things are different, he wants all substrings of hi
s string s to be distinct. Substring is a string formed by some number of co
nsecutive characters of the string. For example, string "aba" has substrings
 "" (empty substring), "a", "b", "a", "ab", "ba", "aba".

    If string s has at least two equal substrings then Kerem will change cha
racters at some positions to some other lowercase English letters. Changing
characters is a very tiring job, so Kerem want to perform as few changes as
possible.

    Your task is to find the minimum number of changes needed to make all th
e substrings of the given string distinct, or determine that it is impossibl
e.

    ...

    """
    pass

# check the correctness of the `main` function
assert
```

Evaluation

Ground Truth Code

```
#!/usr/bin/env python3
from collections import Counter
try:
    while True:
        n = int(input())
        s = input()
        if n > 26:
            print(-1)
        else:
            c = Counter(s)
            print(sum(c.values()) - len(c))
except EOFError:
    pass
```

Output: Generated Test Cases

PyTester — Syntax Correct, Executable

```
assert test_main('2\naa') == '1'
assert test_main('4\nkoko') == '2'
assert test_main('5\nmurat') == '0'
```

GPT3.5 — Syntax Correct, Non-executable

```
assert test_main('2\naa\n') == '1\n'
assert test_main('4\nkoko\n') == '2\n'
assert test_main('5\nmurat\n') == '0\n'
```

StarCoder — Syntax Correct, Non-executable

```
assert  test_main("aa") == "1"
assert  test_main("koko") == "2"
assert  test_main("murat") == "0"
```

Copilot — Syntax Correct, Non-executable

```
assert test_main('2\naa\n') == '1'
assert test_main('4\nkoko\n') == '2'
assert test_main('5\nmurat\n') == '0'
```

InCoder — Syntax incorrect, Non-executable

```
assert  main(main(main(main(main(main(main
(main(main(main(main(main(main(main(main(main
(main(main(main(main(main(main(main(main(main
(main(main(main(main(main(main(main(main(main
(main(main(main(main(main(main(main(solution(
main(main(main(main(main(main(main(main(solut
ion(mains(main(main(main(main(mains(main(main
(main(main(main(solution(mains(main(main(main
(main(main(solution(mains(main(main(main(
```

CodeT5-large — Syntax incorrect, Non-executable

```
assert test_main('2\naa') == '1'
assert test_main('4\nkoko') == '2'
assert test_main('5\nmurat') == '0'
assert test_main('1\nb') == '0'
assert test_main('10\naabba') == '-1'
assert test_main('100\naabbaabbaabbaabbaabbaabba
abbaabbaabbaabbaabbaabbaabbaabbaabbaabbaabbaabba
abbaabbaaba') == '-1'
assert test_main('1\nb') == '0'
assert test_main('1\nb') == '0'
assert test_main('2\nba') == '1'
assert test_main('2\nba') == '1'
assert test_main('3\naz') == '1'
assert test_main('3\naz') == '1'
assert test_main('4\naz') == '2'
assert test_main('5\nkoko') == '2'
assert test_main('6\nmurat') == '0'
assert test_main('7\nmurat') == '0'
assert test_main('8\nkoko') == '0
```

Fig. 4. An example of test cases generated by PyTester and other baseline approaches (depicted from Problem ID #569 of the testing dataset).

Table 5
(RQ2) The performance of our PyTester for the 7 variations of the reward functions. S: Syntax, E: Executability, C: Coverage.

Reward function	Syntax correctness	Passing test case	Code coverage	Mutation score	TestQuality
S-only	99.39%	82.72%	79.47%	60.96%	69.00%
E-only	96.26%	75.73%	72.95%	56.34%	63.58%
C-only	98.59%	**86.87%**	**81.96%**	56.47%	66.87%
S+E	98.80%	83.49%	80.34%	61.25%	69.51%
S+C	**99.42%**	84.41%	80.98%	**61.45%**	**69.87%**
E+C	94.57%	67.78%	65.37%	51.01%	57.31%
S+E+C	98.80%	81.68%	78.34%	58.76%	67.15%

function—i.e., such feedback will greatly help the agent to generate test cases that are syntactically correct during the policy optimization step, which significantly improving the performance. The finding also suggests that syntax correctness must be the minimum consideration when designing a reward function for deep reinforcement learning in the text-to-test case generation task.

In addition, we also find that both Syntax+Coverage and Syntax+Executability mostly perform best (i.e., highly correlated). For Syntax+Coverage, the model aims to directly generate syntactically correct test cases that achieve high code coverage, enabling the generated test cases to be executable against the ground truth code. On the other hand, for Syntax+Executability, the model aims to directly generate syntactically correct test cases that are executable against the ground truth code, enabling the generated test cases to achieve high code coverage. Therefore, considering either Syntax+Coverage or Syntax+Executability in the reward function is the best. Nevertheless, the lowest-performing variation of our PyTester still outperforms most of the LLMs.

(RQ3) what types of runtime errors occur in the pytester-generated test cases?

Results. **Among the incorrectly generated test cases, PyTester mostly encounters an Assertion Error issue for 10.75%.** Despite PyTester performing the best when compared to the studied LLMs, we

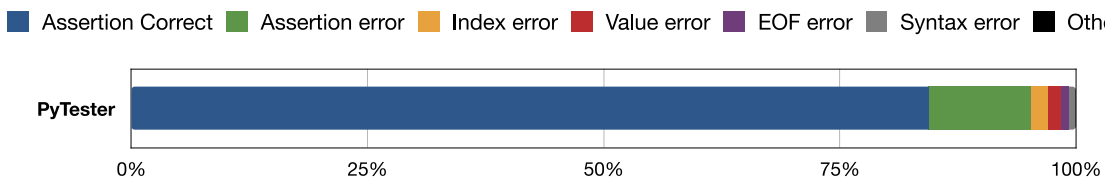

Fig. 5. (RQ3) The common types of runtime errors that occur in the PyTester-generated test cases.

want to gain deeper insights into the incorrectly generated test cases. As mentioned in Table 4, we find that 84.41% of the test cases generated from PyTester are passed (the blue color in Fig. 5), meaning that 15.59% of the generated test cases are not executable, thus incorrect. Thus, we conduct an analysis to investigate what are the common types of runtime errors that occur in the PyTester-generated test cases. As shown in Fig. 5, PyTester encounters the following top five common types of errors as follows:

- Assertion Error (10.75%) is an exception that is raised when an assert statement fails. The AssertionError is raised when the actual output (i.e., the output of the generated assertion) does not match the expected output (i.e., the output from the ground truth code). This finding highlights that as low as 10.75% of the generated test cases are not passed the ground truth code. Therefore, future researchers should consider developing a novel mechanism integrated into the RL framework to improve the passing test case.
- Index Error (1.75%) is an exception that occurs when trying to access an element from a sequence (such as a list, tuple, or string) using an index that is outside the valid range of indices for that sequence. For example, in Problem ID #456, the model should generate the test case as "`assert test_main('13\n o gogmgogogogo')== '***gmg***'`", but the model incorrectly generates the test case as "`assert test_main('13\ngogmgogogogo') == '***gmg***'`". The highlighted character indicates the missing character that is not generated by PyTester, leading to the incorrect access of the index in the sequence, thus, throwing an IndexError exception.
- Value Error (1.35%) is an exception that occurs when a function or operation receives an input that has the correct data type but is not in the expected range or does not have a valid value. For example, in Problem ID #911, the model should generate the test case as "`assert test_main(' 3 2 \n50 85 250\n10 15 25') == 'Limak'`", but the model incorrectly generates the test case as "`assert test_main(' 3 \n2\n1 1\n2 3') == '2\n0 points'`". The Value Error is related to the highlighted character where it should receive two input values, but one of the values is missing, thus, throwing a ValueError exception.
- EOF Error (0.98%) is an exception that occurs when a built-in function like `input()` or a method like `readline()` of a file object encounters an unexpected end of file (EOF) condition while trying to read input or data. For example, in Problem ID #1152, the model should generate the test case as "`test_main('3 3\n0 1 0\n0 1 0\n1 0 0\n1 0 0\n1 0 0\n 1 0 0 ') == 'Yes'`", but the model incorrectly generated the test case as "`test_main('3 3\n0 1 0\n0 1 0\n1 0 0\n1 0 0\n1 0 0') == 'Yes'`". The EOF Error is related to one missing line (highlighted), thus, throwing an EOFError exception.
- Syntax Error (0.58%) is an exception that occurs when the interpreter encounters a statement that does not conform to the rules of the Python programming language. For example, in Problem ID #1184, the model incorrectly generates an incompleted assert statement as "`assert test_main('[a, a, a, a, a, a, a`", throwing a SyntaxError exception. However, such Syntax Error is relatively low when compared to other types of errors. This highlights the strengths of our approach that considers syntax correctness in the reward function, optimizing the models to generate more syntactically correct test cases.

6. Discussion

In this section, we discuss the additional evaluations of PyTester in zero-shot learning on HumanEval (Chen et al., 2021) and MBPP (Austin et al., 2021) datasets. The objective of this analysis is to assess PyTester on the unseen datasets. Notably, the HumanEval and MBPP datasets are relatively small, with sample sizes of $n = 164$ and $n = 257$, respectively.

On the HumanEval dataset, the raw output of PyTester ranks the second place in generating executable test cases with high code coverage, following only GPT-3.5.

Table 6 shows the results of PyTester compared to state-of-the-art LLMs and Pynguin on the HumanEval dataset. Although performing in the zero-shot setting, the results show that PyTester can generate syntactically correct, executable test cases with high code coverage comparable to LLMs such as StarCoder and Copilot. In fact, our PyTester model achieves the second rank in the raw evaluation, following only the GPT-3.5 model. It is important to note that albeit providing the necessary information (i.e., description text, code, and test cases), the HumanEval dataset typically provides a *concise docstring* for a function, different to the APPS dataset (i.e., the training dataset) that contains a *lengthy, detailed description* for a function. For example, a textual description for task ID 0 in HumanEval (i.e., `has_close_elements` function) is *"Check if in a given list of numbers, are any two numbers closer to each other than a given threshold"*.. This example demonstrates that the input of the HumanEval dataset is significantly shorter than the APPS dataset (see example in Fig. 4), which is the training dataset of PyTester. Therefore, this difference presents limitations that can impact the performance of PyTester.

On the MBPP dataset, PyTester performance is comparable to the StarCoder Model.

Table 7 shows the results of PyTester compared to state-of-the-art LLMs and Pynguin on the MBPP dataset. Similar to the HumanEval dataset, the MBPP dataset has a unique nature of the text description, which is significantly different to the APPS dataset (i.e., the training dataset). Specifically, the MBPP dataset provides only a *single-line description* of a function, different to the APPS dataset that contains a *lengthy, detailed description* for a function. For example, a textual description for task ID 11 in MBPP (i.e., `remove_Occ` function) is *"Write a python function to remove the first and last occurrence of a given character from the string"*.. Similarly, the example demonstrates that the input of the MBPP dataset is also significantly shorter with less information than the APPS dataset (Fig. 4). Therefore, this characteristic greatly impacts the performance of our PyTester that trained on a detailed description, resulting in the decline of PyTester's performance compared to the findings on APPS. However, the results in Table 7 still show that our PyTester can perform better than the Finetune CodeT5-large and the InCoder models, and comparable if not better than the significantly larger model such as the StarCoder model.

Lastly, it is worth noting that the input of the Pynguin baseline is a source code (Code-to-Testcase), different to other methods that rely on a textual description (Text-to-Testcase). Consequently, the high scores of passing test cases and code coverage are expected due to the direct exposure to the code.

Table 6
The performance of PyTester in zero-shot learning on HumanEval when compared to state-of-the-art LLMs and Pynguin.

Evaluation setting	Model	Model parameters	Syntax correctness	Passing test case	Code coverage	Mutation score
Raw	**PyTester**	770M	**99.39%**	83.54%	80.09%	48.74%
	Finetuned CodeT5-large	770M	96.34%	68.90%	68.02%	49.80%
	InCoder	6B	12.20%	4.27%	4.16%	4.10%
	StarCoder	15.5B	85.37%	78.05%	77.25%	63.75%
	GPT3.5	>175B	**99.39% (0.61%)**	**89.51% (0.80%)**	**88.36% (0.96%)**	**69.59% (0.76%)**
	Copilot	>12B	83.78% (0.92%)	79.51% (0.70%)	79.00% (0.69%)	63.50% (0.49%)
	Pynguin	–	86.59%	83.54%	82.87%	25.59%
3-test	**PyTester**	770M	99.39%	83.54%	80.11%	48.74%
	Finetuned CodeT5-large	770M	**100.00%**	73.17%	71.67%	50.58%
	InCoder	6B	54.88%	14.63%	14.28%	11.80%
	StarCoder	15.5B	**100.00%**	85.37%	83.99%	65.82%
	GPT3.5	>175B	99.51% (0.51%)	**92.80% (1.00%)**	**91.19% (1.27%)**	**70.62% (1.15%)**
	Copilot	>12B	98.78% (0.00%)	89.63% (0.43%)	88.62% (0.43%)	70.52% (0.25%)
	Pynguin	–	87.80%	85.98%	85.21%	23.61%
5-test	**PyTester**	770M	99.39%	83.54%	80.11%	48.74%
	Finetuned CodeT5-large	770M	99.39%	69.51%	68.63%	50.28%
	InCoder	6B	46.34%	7.93%	7.80%	7.17%
	StarCoder	15.5B	**100.00%**	80.49%	79.65%	64.35%
	GPT3.5	>175B	99.51% (0.51%)	**89.87% (1.26%)**	**88.68% (1.42%)**	**69.62% (0.89%)**
	Copilot	>12B	97.32% (0.33%)	85.12% (0.33%)	84.51% (0.33%)	68.07% (0.23%)
	Pynguin	–	85.37%	84.15%	83.43%	21.45%
Filtered by Syntax Correctness	**PyTester**	770M	98.78%	83.54%	80.11%	48.74%
	Finetuned CodeT5-large	770M	**100.00%**	68.90%	68.04%	49.80%
	InCoder	6B	35.37%	4.27%	4.16%	4.10%
	StarCoder	15.5B	**100.00%**	82.32%	81.54%	67.30%
	GPT3.5	>175B	99.76% (0.33%)	**89.75% (0.80%)**	**88.60% (0.98%)**	**69.81% (0.95%)**
	Copilot	>12B	99.39% (0.00%)	83.54% (0.61%)	83.01% (0.61%)	66.50% (0.52%)
	Pynguin	–	86.59%	83.54%	82.87%	25.59%

Table 7
The performance of PyTester in zero-shot learning on MBPP when compared to state-of-the-art LLMs and Pynguin.

Evaluation setting	Model	Model parameters	Syntax correctness	Passing test case	Code coverage	Mutation score
Raw	**PyTester**	770M	90.66%	15.18%	14.95%	5.74%
	Finetuned CodeT5-large	770M	88.72%	2.72%	2.66%	0.70%
	InCoder	6B	27.63%	2.33%	2.31%	1.48%
	StarCoder	15.5B	43.19%	9.73%	9.62%	5.20%
	GPT3.5	>175B	**98.05% (1.48%)**	37.04% (1.54%)	36.78% (1.23%)	**21.97% (1.30%)**
	Copilot	>12B	61.56% (1.39%)	28.56% (1.05%)	28.54% (1.05%)	16.46% (0.44%)
	Pynguin	–	89.49%	**79.77%**	**75.86%**	20.71%
3-test	**PyTester**	770M	90.66%	15.18%	14.95%	5.74%
	Finetuned CodeT5-large	770M	98.44%	4.67%	4.61%	1.87%
	InCoder	6B	61.87%	2.72%	2.69%	1.48%
	StarCoder	15.5B	87.94%	17.51%	17.42%	9.41%
	GPT3.5	>175B	98.44% (1.46%)	38.60% (1.68%)	38.24% (1.50%)	**22.76% (0.89%)**
	Copilot	>12B	**98.52% (0.17%)**	38.68% (0.59%)	38.58% (0.59%)	21.61% (0.43%)
	Pynguin	–	91.44%	**80.16%**	**76.67%**	19.02%
5-test	**PyTester**	770M	90.66%	15.18%	14.95%	5.74%
	Finetuned CodeT5-large	770M	95.33%	3.11%	3.05%	1.09%
	InCoder	6B	61.87%	2.33%	2.31%	1.48%
	StarCoder	15.5B	78.99%	12.84%	12.73%	6.57%
	GPT3.5	>175B	**98.05% (1.48%)**	37.04% (1.54%)	36.78% (1.23%)	**21.97% (1.20%)**
	Copilot	>12B	95.85% (0.39%)	34.71% (0.70%)	34.67% (0.70%)	20.58% (0.42%)
	Pynguin	–	90.66%	**80.16%**	**77.00%**	21.71%
Filtered by Syntax Correctness	**PyTester**	770M	89.49%	15.18%	14.95%	5.74%
	Finetuned CodeT5-large	770M	99.22%	2.72%	2.66%	0.70%
	InCoder	6B	32.30%	2.33%	2.31%	1.48%
	StarCoder	15.5B	91.44%	15.56%	15.46%	7.74%
	GPT3.5	>175B	98.36% (0.51%)	37.04% (1.54%)	36.78% (1.23%)	**21.90% (1.09%)**
	Copilot	>12B	**100.00% (0.00%)**	35.25% (0.65%)	35.23% (0.65%)	19.80% (0.28%)
	Pynguin	–	89.49%	**79.77%**	**75.86%**	20.71%

7. Related works

In this section, we highlight the difference in our work to the literature on reinforcement learning in software engineering.

Reinforcement learning (RL) (Sutton and Barto, 2018) is a machine-learning approach where an agent learns to map observations to actions by maximizing numerical reward functions. Recently, RL has been applied to text generation tasks. Specifically, LLMs utilize RL to train models based on user preferences through human feedback (RLHF) (Christiano et al., 2017). Consequently, LLMs can exhibit complex and coherent behaviors such as instruction following (Ouyang et al., 2022), summarization (Stiennon et al., 2020), and conversation (Jaques et al., 2019).

In software engineering, reinforcement learning has been adopted in the code generation tasks (Wang et al., 2022; Le et al., 2022) to improve the compilability and execution rate of the generated code. For example, Wang et al. (2022) proposed CompCoder, an RL approach that considers compiler feedback in the reward function to generate more compilable code. Le et al. (2022) proposed CodeRL, an actor-critic (Bahdanau et al., 2016; Konda and Tsitsiklis, 1999) deep reinforcement learning framework for code generation.

Different from prior studies, we formulate the Text-to-Testcase task as a deep reinforcement learning problem. We incorporate the domain knowledge such as test case characteristics (e.g., syntax and code coverage scores) into the model through a reward function. Additionally, we perform a deep analysis of ablation studies on reward components, showing the importance of the test case characteristics. Finally, the results demonstrate that our PyTester outperforms other LLMs, highlighting the novelty and the significance of our work that contributes to the area of Text-to-Testcase generation.

8. Threat to validity

Threats to construct validity. The policy training and the policy optimization steps of our PyTester involve hyperparameter settings (see Table 2 and Table 3). Prior studies raised concerns that different hyperparameter settings may impact the performance of the Deep RL models (Kiran and Ozyildirim, 2022). However, the parameter optimization is beyond the scope of this paper. Thus, future research can consider exploring the impact of the hyperparameter settings on the Deep RL models.

Data leakage is often a critical concern when applying large language models (LLMs) in software engineering (i.e., the phenomenon when data samples in the testing dataset were included in the training dataset). With PyTester, it is based on CodeT5-large, which is pre-trained on the CodeSearchNet dataset, and then later fine-tuned on the APPS benchmark dataset. Prior to the pre-training and fine-tuning step, we carefully analyzed and ensured that none of the testing samples in the APPS dataset appeared in the CodeSearchNet (Husain et al., 2019) and the training set of the APPS dataset to avoid any data leakage problem. Thus, we confirm that the results of our PyTester will not be impacted by the data leakage. With GPT-3.5 (baseline comparison), we are unable to determine if there is any data leakage during the model training of GPT-3.5 due to the close-source nature of GPT-3.5. Despite the risk of data leakage with GPT-3.5, our experiments still confirm that PyTester (with no risk of data leakage) outperforms GPT-3.5 (with the risk of data leakage) for all evaluation measures.

Threats to internal validity. The variation of the reward function may impact the performance of PyTester. However, our RQ2 confirms that, among the 7 variations, the performance varies between 57.31% and 69.87% of the TestCaseScore. Despite the high difference in the performance among the reward functions, we find that the lowest-performing variation of our PyTester still outperforms most of the LLMs. Thus, the reward function does not pose a threat to the validity of our study. Nevertheless, our study is limited by one scheme of reward points (ranging from −2 to 4). However, finding an optimal scheme of reward points is beyond the scope of this work. Thus, future research is encouraged to explore the impact of the reward points on the performance of Deep RL models.

The primary goal of software testing is to identify defects and vulnerabilities within the software. Consequently, automated test case generation is expected to produce test cases that can find bugs accurately. To evaluate this bug-finding capability of a model, a dataset containing buggy code is typically required. However, due to the unavailability of the bug information in our APPS benchmark dataset, it is challenging to directly assess the bug-finding capability of PyTester. To address this challenge, we include the evaluation of the mutation testing to our paper. Mutation testing works by introducing small bugs into the correct code (e.g., replacing a minus operator with a plus operator). If the test cases still pass the mutated code, they will receive a zero Mutation Score, indicating that the test cases fail to identify bugs in the mutated code. This metric provides valuable insight into how well and how sensitive the generated test cases can identify bugs, serving as an alternative means of assessing the potential capability of a model in bug-finding.

Threats to External Validity. We acknowledge that our PyTester is currently limited to the generated test cases based on the textual description of one single call. In real-world scenarios, test cases are more complex (including test prefixes, etc.). Thus, future research may consider the generation of more complex test cases that include test prefixes, etc.

PyTester training dataset is limited to APPS—the only Text-to-Testcase benchmark dataset with sufficient size. In addition to broadening our analysis and ensuring the performance of PyTester, we provide additional evaluations in zero-shot learning on HumanEval and MBPP datasets. Future research may consider increasing the generalizability of PyTester approach with other datasets, programming languages, and contexts.

9. Conclusion

In conclusion, we propose PyTester, a Text-to-Testcase generation approach for Python using Deep Reinforcement Learning via a reward function that considers three types of feedback, namely, syntax correctness, test executability, and code coverage. Through a comprehensive experiment on the APPS benchmark dataset (Hendrycks et al., 2021), we find that:

- Of the test cases generated by PyTester, 99% are syntactically correct, 84% are passed while achieving a code coverage of 80% and a mutation score of 61%, which outperforms all of the studied Large Language Models.
- The top-3 reward functions for our PyTester are the Syntax+Coverage, Syntax+Executability, and Syntax-only types of feedback.
- As low as 15.59% of the PyTester-generated test cases are incorrect. Among the incorrectly generated test cases, our PyTester mostly encounters an Assertion Error issue for 10.75%, followed by other types of errors (i.e., IndexError, ValueError, EOFError, SyntaxError).

Based on these results, we draw the following implications for future research.

- **The performance of smaller language models is not necessarily inferior to that of large language models if carefully designed.** Training or finetuning large language models often requires significant computational resources, including powerful hardware and large datasets, with the assumption that more resources and more datasets will contribute to performance improvement (OpenAI, 2023; Guo et al., 2023). This paper takes a different direction from the existing work by using Deep RL to

enable smaller language models, with the smallest model parameter size (770M) and the fastest inference times by *at least* one order of magnitude, to outperform much larger language models (175B). This finding suggests that future research could consider improving small over large LMs for better resource efficiency.

- **The SE domain knowledge should be considered when designing a deep reinforcement learning architecture.** The performance of many AI models often relies on its model architecture. Thus, many prior studies often adopted various data mining and optimization techniques to improve such AI models. Different from existing work, this paper takes an alternative direction by integrating the SE domain knowledge when designing the reinforcement learning architecture (i.e., incorporating test case characteristics into the reward function). This enables PyTester to generate test cases that are more syntactically correct, executable, complete, and effective while being aligned with a given description.

CRediT authorship contribution statement

Wannita Takerngsaksiri: Writing – review & editing, Writing – original draft, Validation, Methodology, Formal analysis, Data curation, Conceptualization. **Rujikorn Charakorn:** Methodology, Investigation, Formal analysis. **Chakkrit Tantithamthavorn:** Writing – review & editing, Supervision, Funding acquisition, Conceptualization. **Yuan-Fang Li:** Writing – review & editing, Supervision, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

Alagarsamy, S., Tantithamthavorn, C., Aleti, A., 2023. A3test: Assertion-augmented automated test case generation. ArXiv preprint arXiv:2302.10352.

Almasi, M.M., Hemmati, H., Fraser, G., Arcuri, A., Benefelds, J., 2017. An industrial evaluation of unit test generation: Finding real faults in a financial application. In: 2017 IEEE/ACM 39th International Conference on Software Engineering: Software Engineering in Practice Track. ICSE-SEIP, IEEE, pp. 263–272.

Ambler, S.W., 2004. The Object Primer: Agile Model-Driven Development with UML 2.0. Cambridge University Press.

Astels, D., 2003. Test Driven Development: A Practical Guide. Prentice Hall Professional Technical Reference.

Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., et al., 2021. Program synthesis with large language models. ArXiv preprint arXiv:2108.07732.

Bahdanau, D., Brakel, P., Xu, K., Goyal, A., Lowe, R., Pineau, J., Courville, A., Bengio, Y., 2016. An actor-critic algorithm for sequence prediction. ArXiv preprint arXiv:1607.07086.

Beck, K., 2022. Test Driven Development: By Example. Addison-Wesley Professional.

Beck, K., Beedle, M., Van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., et al., 2001. Manifesto for Agile Software Development. Snowbird, UT.

Blasi, A., Goffi, A., Kuznetsov, K., Gorla, A., Ernst, M.D., Pezzè, M., Castellanos, S.D., 2018. Translating code comments to procedure specifications. In: Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis. pp. 242–253.

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J.D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al., 2020. Language models are few-shot learners. Adv. Neural Inf. Process. Syst. 33, 1877–1901.

Cai, X., Lyu, M.R., 2005. The effect of code coverage on fault detection under different testing profiles. In: Proceedings of the 1st International Workshop on Advances in Model-Based Testing. pp. 1–7.

Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H.P.d., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al., 2021. Evaluating large language models trained on code. ArXiv preprint arXiv:2107.03374.

Chen, B., Zhang, F., Nguyen, A., Zan, D., Lin, Z., Lou, J.-G., Chen, W., 2022. Codet: Code generation with generated tests. ArXiv preprint arXiv:2207.10397.

Christiano, P.F., Leike, J., Brown, T., Martic, M., Legg, S., Amodei, D., 2017. Deep reinforcement learning from human preferences. Adv. Neural Inf. Process. Syst. 30.

Dinella, E., Ryan, G., Mytkowicz, T., Lahiri, S.K., 2022. Toga: A neural method for test oracle generation. In: Proceedings of the 44th International Conference on Software Engineering. pp. 2130–2141.

Erdogmus, H., Morisio, M., Torchiano, M., 2005. On the effectiveness of the test-first approach to programming. IEEE Trans. Softw. Eng. 31 (3), 226–237.

Fischbach, J., Frattini, J., Vogelsang, A., Mendez, D., Unterkalmsteiner, M., Wehrle, A., Henao, P.R., Yousefi, P., Juricic, T., Radduenz, J., et al., 2023. Automatic creation of acceptance tests by extracting conditionals from requirements: NLP approach and case study. J. Syst. Softw. 197, 111549.

Fraser, G., Arcuri, A., 2011. Evosuite: automatic test suite generation for object-oriented software. In: Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering. pp. 416–419.

Fried, D., Aghajanyan, A., Lin, J., Wang, S., Wallace, E., Shi, F., Zhong, R., Yih, W.-t., Zettlemoyer, L., Lewis, M., 2022. Incoder: A generative model for code infilling and synthesis. ArXiv preprint arXiv:2204.05999.

GitHub, 2021. Your AI pair programmer. URL: https://github.com/features/copilot.

Grano, G., Palomba, F., Di Nucci, D., De Lucia, A., Gall, H.C., 2019. Scented since the beginning: On the diffuseness of test smells in automatically generated test code. J. Syst. Softw. 156, 312–327.

Guo, Q., Cao, J., Xie, X., Liu, S., Li, X., Chen, B., Peng, X., 2023. Exploring the potential of ChatGPT in automated code refinement: An empirical study. ArXiv preprint arXiv:2309.08221.

Hemmati, H., 2015. How effective are code coverage criteria? In: 2015 IEEE International Conference on Software Quality, Reliability and Security. IEEE, pp. 151–156.

Hendrycks, D., Basart, S., Kadavath, S., Mazeika, M., Arora, A., Guo, E., Burns, C., Puranik, S., He, H., Song, D., et al., 2021. Measuring coding challenge competence with apps. ArXiv preprint arXiv:2105.09938.

Hou, X., Zhao, Y., Liu, Y., Yang, Z., Wang, K., Li, L., Luo, X., Lo, D., Grundy, J., Wang, H., 2023. Large language models for software engineering: A systematic literature review. ArXiv preprint arXiv:2308.10620.

Husain, H., Wu, H.-H., Gazit, T., Allamanis, M., Brockschmidt, M., 2019. Codesearchnet challenge: Evaluating the state of semantic code search. ArXiv preprint arXiv:1909.09436.

Janzen, D., Saiedian, H., 2008. Does test-driven development really improve software design quality? IEEE Softw. 25 (2), 77–84.

Jaques, N., Ghandeharioun, A., Shen, J.H., Ferguson, C., Lapedriza, A., Jones, N., Gu, S., Picard, R., 2019. Way off-policy batch deep reinforcement learning of implicit human preferences in dialog. ArXiv preprint arXiv:1907.00456.

Jia, Y., Harman, M., 2010. An analysis and survey of the development of mutation testing. IEEE Trans. Softw. Eng. 37 (5), 649–678.

Kingma, D.P., Ba, J., 2014. Adam: A method for stochastic optimization. ArXiv preprint arXiv:1412.6980.

Kiran, M., Ozyildirim, M., 2022. Hyperparameter tuning for deep reinforcement learning applications. ArXiv preprint arXiv:2201.11182.

Konda, V., Tsitsiklis, J., 1999. Actor-critic algorithms. Adv. Neural Inf. Process. Syst. 12.

Kullback, S., Leibler, R.A., 1951. On information and sufficiency. Ann. Math. Stat. 22 (1), 79–86.

Le, H., Wang, Y., Gotmare, A.D., Savarese, S., Hoi, S.C.H., 2022. Coderl: Mastering code generation through pretrained models and deep reinforcement learning. Adv. Neural Inf. Process. Syst. 35, 21314–21328.

Li, R., Allal, L.B., Zi, Y., Muennighoff, N., Kocetkov, D., Mou, C., Marone, M., Akiki, C., Li, J., Chim, J., et al., 2023. StarCoder: may the source be with you!. ArXiv preprint arXiv:2305.06161.

Lukasczyk, S., Fraser, G., 2022. Pynguin: Automated unit test generation for python. In: Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings. pp. 168–172.

Madeyski, L., 2010. The impact of test-first programming on branch coverage and mutation score indicator of unit tests: An experiment. Inf. Softw. Technol. 52 (2), 169–184.

Mueller, M.M., Hagner, O., 2002. Experiment about test-first programming. IEE Proc. Softw. 149 (5), 131–136.

OpenAI, 2023. GPT-4 technical report. arXiv:2303.08774.

Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., et al., 2022. Training language models to follow instructions with human feedback. Adv. Neural Inf. Process. Syst. 35, 27730–27744.

Pacheco, C., Ernst, M.D., 2007. Randoop: feedback-directed random testing for java. In: Companion To the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems and Applications Companion. pp. 815–816.

Pacheco, C., Lahiri, S.K., Ernst, M.D., Ball, T., 2007. Feedback-directed random test generation. In: ICSE 2007, Proceedings of the 29th International Conference on Software Engineering. Minneapolis, MN, USA, pp. 75–84.

Palomba, F., Di Nucci, D., Panichella, A., Oliveto, R., De Lucia, A., 2016. On the diffusion of test smells in automatically generated test code: An empirical study. In: Proceedings of the 9th International Workshop on Search-Based Software Testing. pp. 5–14.

Papadakis, M., Kintis, M., Zhang, J., Jia, Y., Le Traon, Y., Harman, M., 2019. Mutation testing advances: an analysis and survey. In: Advances in Computers. vol. 112, Elsevier, pp. 275–378.

Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., Liu, P.J., 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. J. Mach. Learn. Res. 21 (1), 5485–5551.

Rozière, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X.E., Adi, Y., Liu, J., Remez, T., Rapin, J., et al., 2023. Code llama: Open foundation models for code. ArXiv preprint arXiv:2308.12950.

Schäfer, M., Nadi, S., Eghbali, A., Tip, F., 2023. An empirical evaluation of using large language models for automated unit test generation. IEEE Transactions on Software Engineering.

Schulman, J., Moritz, P., Levine, S., Jordan, M., Abbeel, P., 2015. High-dimensional continuous control using generalized advantage estimation. ArXiv preprint arXiv:1506.02438.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O., 2017. Proximal policy optimization algorithms. ArXiv preprint arXiv:1707.06347.

Shen, Z., Liu, J., He, Y., Zhang, X., Xu, R., Yu, H., Cui, P., 2021. Towards out-of-distribution generalization: A survey. ArXiv preprint arXiv:2108.13624.

Siddiq, M.L., Santos, J., Tanvir, R.H., Ulfat, N., Rifat, F.A., Lopes, V.C., 2023. Exploring the effectiveness of large language models in generating unit tests. ArXiv preprint arXiv:2305.00418.

Stiennon, N., Ouyang, L., Wu, J., Ziegler, D., Lowe, R., Voss, C., Radford, A., Amodei, D., Christiano, P.F., 2020. Learning to summarize with human feedback. Adv. Neural Inf. Process. Syst. 33, 3008–3021.

Sutton, R.S., Barto, A.G., 2018. Reinforcement Learning: An Introduction. MIT Press.

Tang, Y., Liu, Z., Zhou, Z., Luo, X., 2023. ChatGPT vs SBST: A comparative assessment of unit test suite generation. ArXiv preprint arXiv:2307.00588.

Tufano, M., Drain, D., Svyatkovskiy, A., Deng, S.K., Sundaresan, N., 2020. Unit test case generation with transformers and focal context. ArXiv preprint arXiv:2009.05617.

Wang, C., Pastore, F., Goknil, A., Briand, L.C., 2020. Automatic generation of acceptance test cases from use case specifications: an nlp-based approach. IEEE Trans. Softw. Eng. 48 (2), 585–616.

Wang, Y., Wang, W., Joty, S.R., Hoi, S.C.H., 2021. CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. In: EMNLP. Association for Computational Linguistics, pp. 8696–8708.

Wang, X., Wang, Y., Wan, Y., Mi, F., Li, Y., Zhou, P., Liu, J., Wu, H., Jiang, X., Liu, Q., 2022. Compilable neural code generation with compiler feedback. ArXiv preprint arXiv:2203.05132.

Watson, C., Tufano, M., Moran, K., Bavota, G., Poshyvanyk, D., 2020. On learning meaningful assert statements for unit test cases. In: Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering. pp. 1398–1409.

Xie, Z., Chen, Y., Zhi, C., Deng, S., Yin, J., 2023. ChatUniTest: a ChatGPT-based automated unit test generation tool. ArXiv preprint arXiv:2305.04764.

Yuan, Z., Lou, Y., Liu, M., Ding, S., Wang, K., Chen, Y., Peng, X., 2023. No more manual tests? Evaluating and improving ChatGPT for unit test generation. ArXiv preprint arXiv:2305.04207.

Ziegler, D.M., Stiennon, N., Wu, J., Brown, T.B., Radford, A., Amodei, D., Christiano, P., Irving, G., 2019. Fine-tuning language models from human preferences. ArXiv preprint arXiv:1909.08593.

Wannita Takerngsaksiri is a Ph.D. candidate at Monash University, Australia. Her research interests encompass automated software development, machine learning (ML), and natural language processing (NLP). She focuses on improving code generation by developing innovative computational methods and advanced AI techniques to enhance the coding process, ultimately making it more effective, practical, and trustworthy for developers.

Rujikorn Charakorn received a B.Eng. in Computer Engineering from Chulalongkorn Universitiy. He is currently a Ph.D. student at the School of Information Science and Technology, Vidyasirimedhi Institute of Science and Technology (VISTEC). His research interests include deep reinforcement learning, cooperative and competitive multi-agent learning and computer vision.

Chakkrit (Kla) Tantithamthavorn is an ARC DECRA Fellow and a Senior Lecturer in Software Engineering in the Faculty of Information Technology, Monash University, Australia. He is pioneering an emerging research area of Explainable AI for Software Engineering (http://xai4se.github.io), inventing many AI-based technologies to improve developers' productivity and make software systems more reliable and more secure, while being explainable to practitioners. To date, the XAI4SE book has attracted 10,000+ page views from 70 countries worldwide. He regularly published at TSE, ICSE, FSE, EMSE, ASE, and MSR, all of which are top software engineering venues. The excellence of his research is recognized through many awards including an ACM SIGSOFT Distinguished Paper Award 2021, an ARC's Discovery Early Career Researcher Award 2020, the World Most Impactful Early-Stage SE Researcher based on a bibliometric assessment of software engineering (2013–2020).

Yuan-Fang Li is an Associate Professor at the Faculty of IT, Monash University. His research interest is artificial intelligence, particularly the intersection between natural language processing and knowledge representation. His recent investigations include the following tasks: (1) neuro-symbolic approaches to complex question answering, (2) knowledge graph construction from text/images, and (3) graph representation learning. His research work has been published at top AI and NLP venues including ACL, EMNLP, ECCV, ICCV, ICLR, NeurIPS, IEEE TNNLS, and Pattern Recognition.